CODE AND BRANCH COVERAGE REPORT

Hospital Management System

1. Introduction

Code coverage is a software testing metric used to measure how much of an application's source code is executed during automated testing. This report presents the line coverage and branch coverage results obtained for the frontend of the Hospital Management System using Vitest and the V8 coverage provider.

2. Objectives

• To measure the percentage of source code statements and lines executed during testing.

• To measure the percentage of decision branches executed during testing.

• To identify source files with uncovered code and branches.

• To evaluate the effectiveness of the current frontend automated test suite.

3. Testing Environment

Component	Details
Operating System	Windows (PowerShell terminal shown in execution evidence)
Programming Language	JavaScript / JSX
Frontend Framework	React 19 with Vite
Testing Framework	Vitest 5.1.0
Coverage Tool	V8 coverage provider (@vitest/coverage-v8)
Application	Hospital Management System – frontend
Test Data / API	API requests mocked in frontend component tests; no database coverage claimed

4. Coverage Measurement

Code and branch coverage were measured using Vitest with the V8 coverage provider. The configured coverage report includes frontend JavaScript and JSX source files and excludes the test setup directory and the application's main entry file.

Command used:

```
npm run test:coverage
```

The command runs the automated frontend test suite and produces statement, branch, function, and line coverage percentages, along with uncovered line numbers.

5. Code Coverage Results

Metric	Result
Statement Coverage	41.69%
Line Coverage	43.22%
Function Coverage	36.55%

The measured line coverage is 43.22%. This means that 43.22% of the instrumented executable lines included in this frontend coverage run were executed by the tests.

6. Branch Coverage Results

Metric	Result
Overall Branch Coverage	38.65%

Branch coverage measures whether alternative outcomes of conditional expressions and decision points are exercised. The run achieved 38.65% branch coverage, indicating that many alternative decision paths remain untested. The supplied V8 terminal summary reports the branch percentage but does not provide total, covered, missed, or partially covered branch counts; those counts are therefore not estimated in this report.

7. Test Execution Summary

Metric	Result
Total Test Files	4
Test Files Passed	4
Total Tests Executed	15
Passed	15
Failed	0
Test Execution Time	2.68 seconds

All 15 automated tests passed successfully across the four test files shown in the terminal output. The overall statement coverage was 41.69%, line coverage was 43.22%, branch coverage was 38.65%, and function coverage was 36.55%.

8. Analysis

Source File	Statements	Branches	Functions	Lines
App.jsx	17.24%	40.29%	11.11%	18.51%
BedAvailability.jsx	94.73%	88.23%	88.23%	96.29%
Booking.jsx	52.25%	36.41%	42.00%	53.73%
DoctorDashboard.jsx	0.68%	0.00%	0.00%	0.76%
MedicinePurchase.jsx	71.42%	64.70%	66.66%	75.00%
All files	41.69%	38.65%	36.55%	43.22%

• **BedAvailability.jsx:** Highest measured coverage, with 96.29% line coverage and 88.23% branch coverage.

• **MedicinePurchase.jsx:** 75.00% line coverage and 64.70% branch coverage; additional tests may exercise remaining conditions and paths.

• **Booking.jsx:** 53.73% line coverage and 36.41% branch coverage; several booking workflows and conditional paths remain uncovered.

• **DoctorDashboard.jsx:** 0.76% line coverage and 0.00% branch coverage, making it the largest identified testing gap.

• **App.jsx:** 18.51% line coverage; much of the component's measured code was not exercised in this test run.

The V8 report listed the following uncovered line ranges: App.jsx — 16–76, 80, 84, 132–184; BedAvailability.jsx — 77, 118; Booking.jsx — 137, 144, 158, 172, 188, 195–284, 288–301, 318–319, 340–341, 358–359, 366–387, 436–437, 466–467, 523–538, 564–855; DoctorDashboard.jsx — 9–486; MedicinePurchase.jsx — 61, 65–72, 83–87, 116–128.

9. Conclusion

Code and branch coverage analysis was successfully performed on the frontend of the Hospital Management System using Vitest 5.1.0 and the V8 coverage provider. The application achieved 43.22% line coverage and 38.65% branch coverage, with all 15 automated tests passing successfully across four test files. The results show that the current test suite exercises selected frontend functionality, while substantial portions of the code and decision paths remain untested. Further test cases should prioritize DoctorDashboard.jsx and Booking.jsx, followed by the remaining uncovered paths in App.jsx and MedicinePurchase.jsx. BedAvailability.jsx currently has the strongest measured coverage.

Appendix A. Test Execution Evidence

The following screenshot is the terminal output supplied for this report. It shows the Vitest run, the four passing test files, all 15 passing tests, and the V8 coverage table.

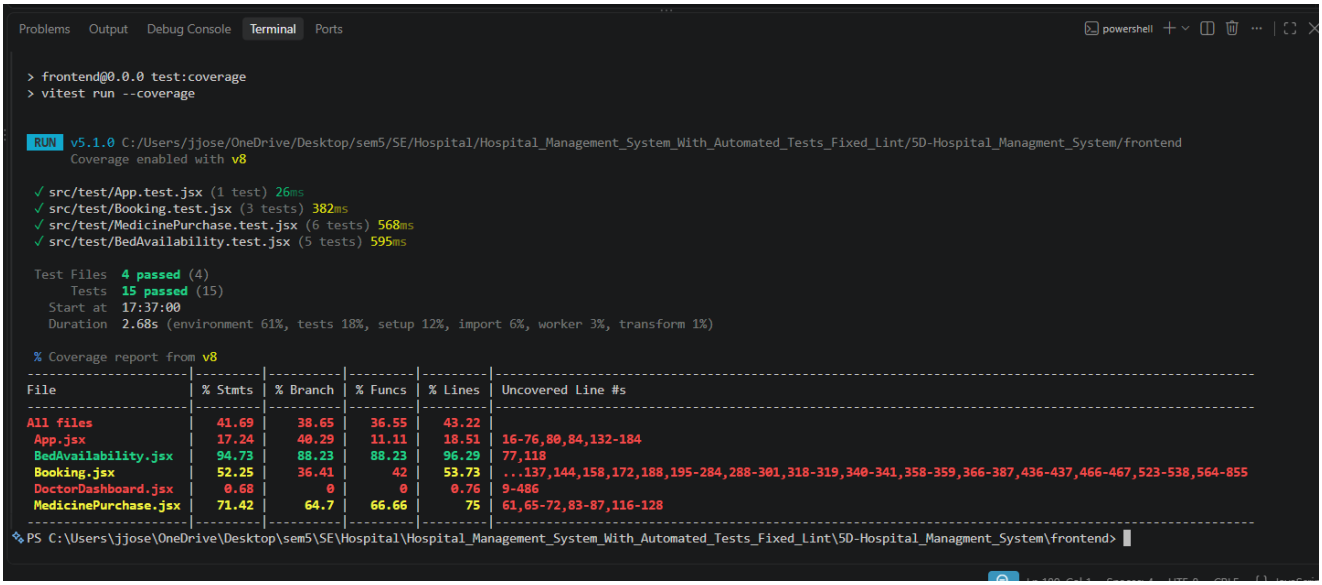


Figure A1. Frontend test execution and V8 coverage output.